

Performance Engineering with a Coding Agent: A Real HPC Case Study

Abstract

"Vibe coding" — letting an AI agent produce software mostly on its own, calling tools, testing, and fixing its own mistakes in a loop, with little human interaction along the way — is everywhere right now. This tutorial looks at a different use of the same AI agents: hands-on practice in data analysis and exploration, and in performance engineering under real hardware constraints on an actual HPC system.

Partial differential equations (PDEs) are a common way to model reality across physics and other fields, and getting a PDE solver to run correctly and effectively on real hardware is one of the most demanding engineering tasks. This tutorial works through one such example: our own implementation of a solver — computing the recombination correction factor for a pulsed proton beam in an ionization chamber — with a physically validated simulation, an analytic correctness check, and a slow, naive implementation to start from.

Participants work on that naive implementation using `opencode`, an AI coding agent that can use several models hosted directly on Cyfronet's infrastructure — including GLM 5.2, whose coding ability is similar to Claude Sonnet's — at no cost to the user. All data and tasks stay within Cyfronet's infrastructure. Three skills matter here: using the agent as a data-analysis tool (profile, plot, and explain a run instead of hand-typing `matplotlib`), directing it toward optimization that is both algorithmic and hardware-aware (using Athena's real core count, cache size, and NUMA layout as constraints, not guesses), and using it safely — permissions, review discipline, and version control, so it can't do real damage.

Agenda

11:00–13:30 — Infrastructure, safety, and diagnosis

- Introduction to the domain problem, and a walkthrough of the naive implementation of the PDE solver.
- Cyfronet-hosted LLMs: why running GLM 5.2 on-premises matters — free at this scale, data never leaves the cluster, coding ability similar to Claude Sonnet's
- Using an agent safely: permission modes, reviewing diffs before accepting them, git commits as checkpoints, never approving destructive commands or job submissions blindly — the rules for the rest of the day
- Setup: Athena account, `opencode` + GLM 5.2 token, a quick "hello agent" test — and the day's submission rule: the agent drafts job scripts, participants review and submit them by hand

- Orientation: the problem we're solving, how the starting code works, and the materials everyone begins with — the code itself, a way to measure performance, and a way to check the results are still correct
- Hands-on: treat the agent as a lab assistant, not an author — "run the sweep, plot wall time and speedup vs. thread count, explain the shape" — and write a short diagnosis based on the output

14:20–16:00 — Optimization: algorithmic and hardware-aware

- Reframing the question: instead of "make it faster," have the agent check the hardware it's running on — cores, cache, NUMA layout.
- Hands-on: algorithmic optimization — restructuring the hot loop
- Hands-on: hardware-aware optimization — deciding how, and whether, to parallelize
- Re-run the timing harness and the correctness check after every change

16:30–18:00 — Solution reveal, security debrief, and wrap-up

- Synthesis: compare each participant's findings against the optimized reference
- Discussion: where the agent's hardware reasoning held up under measurement, where it didn't, and how participants told the difference
- Security retrospective: LLMs executing risky commands, why the rules held, and what to take away for using coding agents on shared HPC accounts
- Wrap-up: agent as an analysis-and-optimization tool, not autocomplete; the case for on-prem LLMs; pointers to the reference repo